

ICEV — INSTITUTO DE ENSINO SUPERIOR

Introdução ao Desenvolvimento de Jogos

VOID DESCENT

Game Design Document

Requisitos GDD — P1

Alunos:

Luis Gustavo Olímpio

Lauan Matheus

José Melquíades

João Leonardi

Teresina — PI

Março de 2026

Sumário

1	Visão Geral (High Concept)	2
2	Mecânicas de Jogo (Game Mechanics)	2
2.1	Controles	2
2.2	Condições de Vitória e Derrota	3
2.3	Sistemas	3
2.3.1	Combate	3
2.3.2	Vida	4
2.3.3	Itens	5
2.3.4	Geração Procedural de Dungeon	6
2.3.5	Pontuação	6
3	O Game Loop	8
3.1	Loop Micro (segundo a segundo)	8
3.2	Loop Macro (minuto a minuto)	10
4	Narrativa e Ambientação	12
4.1	Sinopse	12
4.2	Personagens	12
4.3	Cenário	13
5	Level Design (Esboço)	14
5.1	Fluxo de Telas	14
5.2	Progressão de Dificuldade	14
5.3	Mockup da Tela	16
6	Estética e Áudio (Direção de Arte)	17
6.1	Estilo Visual	17
6.1.1	Moodboard	18
6.2	Sonoplastia	19

7	Planejamento Técnico	20
7.1	Equipe	20
7.2	Engine e Tecnologia	20
7.3	Arquitetura de Componentes	21
7.4	Ferramentas de Apoio	21
7.5	Cronograma Macro	21
7.6	Dificuldades Técnicas Previstas	22
7.7	Posicionamento no Mercado	23
A	Histórico de Revisões	24

1. Visão Geral (High Concept)

Título do Jogo: VOID DESCENT

Elevator Pitch: “Roguelike onde cada morte recria o labirinto. Desça três andares de um abismo procedural, colete armas e destrua o Núcleo antes que o Void te consuma.”

Gênero e Plataforma: Trata-se de um roguelike top-down em tempo real, desenvolvido para PC (Windows). A base tecnológica é JavaScript puro com renderização WebGL, e a versão final será distribuída como executável `.exe` por meio do Electron, o que elimina a necessidade de o jogador abrir um navegador para jogar e garante uma experiência nativa de desktop.

Público-Alvo: O público primário são jogadores entre 16 e 30 anos que já consomem roguelikes e shooters top-down com frequência, e que valorizam a rejogabilidade e o domínio progressivo de mecânicas através da repetição. O formato de runs curtas, com duração entre 8 e 12 minutos por partida, também abre espaço para um público secundário de jogadores mais casuais, que buscam sessões rápidas e intensas sem compromisso de longo prazo com um único save ou campanha.

Diferencial (USP): O principal diferencial de Void Descent é a proposta de ser inteiramente procedural. Na prática, isso significa que toda a arte, todos os mapas, todos os inimigos e todo o áudio são gerados por algoritmos em tempo de execução, sem depender de nenhum asset externo. Não há texturas desenhadas à mão, não há sprites importados de bancos de imagem, não há arquivos de som gravados. O estilo visual adota uma estética neon-sobre-preto com blocos 3D, sprites procedurais, iluminação por vértice, fog atmosférico e posterização de cor, o que produz uma atmosfera densa e imersiva mesmo com custo técnico baixo. Além disso, como cada dungeon é regenerada integralmente a cada nova run, não existem duas partidas iguais, e a rejogabilidade surge de forma orgânica, como consequência direta da arquitetura procedural do jogo.

2. Mecânicas de Jogo (Game Mechanics)

2.1. Controles

O esquema de controle adotado é o twin-stick adaptado para teclado e mouse, que é o padrão consolidado em shooters top-down e foi escolhido justamente por oferecer separação total entre movimentação e mira. Na prática, o jogador usa WASD para se deslocar pelo cenário e o mouse para direcionar a mira e os ataques. Essa separação é

fundamental porque permite que o jogador se mova em uma direção enquanto dispara em outra, o que abre espaço para manobras evasivas e posicionamento tático durante o combate.

Tabela 1 – Mapeamento de controles

Tecla	Ação
W / A / S / D	Movimentação em 4 direções e diagonais
Mouse (cursor)	Direção da mira e dos projéteis
Clique esquerdo	Atacar (melee ou ranged, conforme arma equipada)
Espaço	Dash (esquiva rápida com invencibilidade breve, cooldown de 1,5 s)
E	Interagir (coletar item ou usar escada)
Esc	Pausar o jogo

2.2. Condições de Vitória e Derrota

A **vitória** é alcançada quando o jogador consegue derrotar o boss “O Núcleo” na arena do Andar 3. Assim que o boss é destruído, o jogo redireciona automaticamente para a Tela de Vitória, onde são exibidas as estatísticas completas daquela run: o tempo total gasto, a quantidade de inimigos eliminados, os itens coletados ao longo do caminho e a pontuação final calculada.

A **derrota**, por sua vez, ocorre no momento em que os pontos de vida do jogador chegam a zero. A transição para a Tela de Game Over é imediata e exibe as mesmas estatísticas da run, além de oferecer o botão “Tentar Novamente”, que reinicia a partida a partir do Andar 1 com uma dungeon completamente regenerada. É importante destacar que o jogo adota o modelo de permadeath: todo o progresso de itens, armas e passivos acumulado ao longo da run é permanentemente perdido. Essa escolha de design é deliberada, pois faz com que cada decisão durante a partida carregue peso real, já que não existe possibilidade de “salvar e carregar”.

2.3. Sistemas

2.3.1. Combate

O combate é inteiramente em tempo real, sem nenhuma pausa, menu de seleção ou sistema de turnos. Essa escolha é proposital: em um combate em tempo real, a habilidade mecânica do jogador (reflexos, mira, posicionamento espacial) é testada de forma contínua e ininterrupta, ao contrário de sistemas baseados em turnos onde a

vitória depende predominantemente de decisões estratégicas isoladas. O jogador pode equipar armas de dois tipos: **melee**, que causa mais dano mas exige proximidade com o alvo, ou **ranged**, que dispara projéteis a distância com dano menor em troca de segurança posicional. Essa distinção não é meramente funcional; ela cria um dilema constante entre risco e recompensa que permeia toda a experiência de combate e se reconfigura a cada sala, dependendo da composição de inimigos encontrada. Uma sala cheia de Seekers (melee, lentos) favorece armas ranged; uma sala com Shooters (ranged, frágeis) pode favorecer uma arma melee rápida que os elimine antes de atirarem. A leitura rápida de cada sala e a adaptação tática instantânea é o que separa jogadores iniciantes de jogadores experientes. Além disso, cada inimigo telegrafia seus ataques com um flash visual antes de efetivamente agir, o que dá ao jogador uma janela de reação curta porém suficiente para quem está atento. Esse telegrama é um pilar do design de combate justo: o jogador nunca é punido por algo que não poderia ter previsto.

O dash é a principal ferramenta defensiva e, argumentavelmente, a mecânica mais importante de todo o jogo. Ao ser acionado, ele concede 0,2s de invencibilidade (conhecidos como *i-frames* na terminologia de game design), o que permite ao jogador atravessar projéteis e ataques inimigos sem receber dano durante essa janela. O cooldown de 1,5s entre usos consecutivos impede que o jogador abuse da mecânica e força uma escolha deliberada sobre o momento certo para desviar: usar o dash cedo demais para evitar um projétil pode significar não tê-lo disponível quando um ataque mais perigoso vier em seguida. Esse gerenciamento de cooldown adiciona uma camada de profundidade estratégica ao que poderia ser uma mecânica puramente reativa. Tocar fisicamente em inimigos também causa dano de contato, punindo posicionamento descuidado e incentivando o jogador a manter distância segura mesmo quando não está sob ataque direto. O resultado é um sistema de combate onde a movimentação é tão importante quanto o ataque: sobreviver não é apenas questão de causar dano rápido, mas de ocupar o espaço certo no momento certo.

2.3.2. Vida

O jogador inicia cada run com 5 corações, sendo que cada coração representa 1 ponto de vida. O número 5 não é arbitrário: com 5 pontos de vida, o jogador pode cometer exatamente 4 erros antes de morrer, o que é suficiente para ser tolerável nas primeiras salas, mas escasso o bastante para criar tensão real nos andares mais profundos, onde a cura acumulada até aquele ponto já foi consumida. Ataques de inimigos comuns retiram 1 coração, enquanto o boss retira 2 por golpe, o que torna os erros contra ele especialmente punitivos e comunica, sem nenhum texto explicativo, que o confronto final exige um nível de atenção e precisão diferente de tudo que veio antes. Os itens de cura existem, mas se tornam progressivamente mais escassos à medida que o jogador

desce: aparecem em baús ou como drops aleatórios após limpar salas, restauram apenas 1 coração por uso e têm frequência de spawn mais alta no Andar 1 do que nos andares seguintes. O sistema não inclui nenhum tipo de regeneração passiva, de modo que cada hit recebido tem peso real e acumulativo na sobrevivência da run: um jogador que chega ao Andar 3 com 2 corações restantes está em situação fundamentalmente diferente de um que chega com 5, e essa diferença é resultado direto da performance nas salas anteriores. Essa escassez é uma decisão de design central: ela força o jogador a tratar HP como um recurso estratégico finito, não como algo que se recupera naturalmente com o tempo, e faz com que cada golpe evitado com sucesso tenha valor tangível na economia interna da run.

2.3.3. Itens

O sistema de itens foi projetado para ser simples de entender mas estratégico de utilizar, seguindo o princípio de *easy to learn, hard to master* que é característico dos melhores roguelikes. Existe um pool fixo de 8 armas predefinidas, cada uma com atributos distintos de dano, velocidade, alcance e tipo (melee ou ranged). A título de exemplo, a “Lâmina Rápida” é uma arma melee com dano 2 e velocidade alta, ideal para jogadores agressivos que preferem eliminar inimigos rapidamente a curta distância, enquanto o “Canhão Neon” é uma arma ranged com dano 4 e velocidade lenta, mais adequada para quem prefere manter distância e trocar DPS (dano por segundo) por segurança posicional. O jogador equipa uma única arma por vez e pode trocá-la ao encontrar outra no chão, descartando a anterior permanentemente. Essa limitação de slot único é uma decisão de design central, pois transforma cada troca de arma numa escolha com consequências reais e irreversíveis: o jogador não pode “testar” a arma nova e voltar atrás, o que força uma avaliação rápida de custo-benefício antes de cada troca.

Além das armas, o jogador dispõe de 1 slot de **passivo**, que é ocupado por modificadores que persistem durante toda a run (como velocidade aumentada, dano extra ou HP adicional), e 1 slot de **consumível**, reservado para poções de cura. A limitação a um único passivo gera o mesmo tipo de dilema das armas: ao encontrar um novo passivo, o jogador precisa avaliar se o benefício oferecido supera o que já possui, sabendo que a troca é definitiva. Baús aparecem uma vez por andar e contêm um item aleatório, funcionando como o principal ponto de recompensa garantida em cada nível e como momento de decisão estratégica, já que o item dentro do baú pode exigir o abandono de algo que o jogador já utiliza.

2.3.4. Geração Procedural de Dungeon

A geração de cada andar é feita por um algoritmo de *room placement*, que funciona da seguinte forma: salas retangulares de tamanhos variados são posicionadas aleatoriamente sobre uma grade bidimensional, respeitando regras de espaçamento mínimo para evitar sobreposição e garantir corredores transponíveis, e em seguida o algoritmo traça corredores para conectá-las. Após essa etapa, uma validação por *flood fill* percorre o mapa a partir do ponto de spawn do jogador para garantir que todas as salas sejam alcançáveis; caso alguma sala fique isolada, o algoritmo descarta o layout e regenera integralmente até obter um resultado válido. Essa abordagem de geração com pós-validação garante que o jogador nunca encontre caminhos bloqueados ou salas inacessíveis, mantendo a integridade do fluxo de gameplay mesmo diante da aleatoriedade inerente ao processo. O Andar 1 contém entre 5 e 6 salas e o Andar 2 entre 6 e 7 salas, ambos incluindo obrigatoriamente uma sala de baú (ver Seção 5.2 para detalhamento da progressão de dificuldade por andar). O Andar 3 é intencionalmente mais curto, composto por poucas salas regulares seguidas diretamente pela arena do boss, de modo a concentrar a tensão no confronto final e evitar que o jogador chegue ao boss com fadiga decisória excessiva.

O fluxo dentro de cada sala segue uma regra simples mas eficaz, comum a roguelikes clássicos: ao entrar em uma sala que contém inimigos, as portas trancam automaticamente e só voltam a abrir quando todos os inimigos daquela sala forem eliminados. Esse mecanismo, conhecido como *lock-and-clear*, cumpre duas funções simultâneas: impede que o jogador evite combates simplesmente correndo pelas salas, e transforma cada sala em uma arena contida com início, meio e fim definidos, o que proporciona uma sensação de conclusão e recompensa a cada desafio superado. A escada de descida para o próximo andar aparece na última sala limpa do andar atual, incentivando o jogador a explorar o nível por completo antes de prosseguir e garantindo que ele enfrente todos os encontros projetados para aquele estágio de dificuldade.

2.3.5. Pontuação

A pontuação de cada run é calculada pela fórmula $\text{score} = \text{kills} \times \text{combo} + \text{itens} \times 100$, onde o multiplicador de combo incrementa a cada eliminação consecutiva que o jogador realiza sem levar dano, até o teto de $\times 4$. Quando o jogador é atingido, o combo reseta imediatamente para $\times 1$, representando não apenas perda de HP mas também perda de potencial de pontuação. A lógica por trás desse sistema é criar uma tensão permanente entre duas estratégias opostas: avançar agressivamente para manter o combo alto e multiplicar os pontos, arriscando levar dano e perder o multiplicador, ou recuar para preservar HP e garantir a sobrevivência, sacrificando a pontuação acumulada. Essa tensão é intencional, pois transforma a pontuação em uma medida

não apenas de eficiência, mas de ousadia calculada. Jogadores que dominam o dash e o posicionamento conseguem manter combos longos e atingir scores significativamente maiores, o que alimenta a competição pessoal por personal bests entre runs e cria uma camada adicional de rejogabilidade: mesmo após completar o jogo (derrotar o boss), o jogador tem motivação para continuar jogando em busca da pontuação perfeita, onde nenhum dano é recebido e o combo nunca é quebrado.

3. O Game Loop

O game loop é, por definição, o ciclo de ações que o jogador repete continuamente ao longo de toda a experiência de jogo. Em termos de design, a qualidade desse ciclo é o que determina se um jogo será intrinsecamente divertido ou apenas tolerável: um loop bem construído gera engajamento autossuficiente, onde o próprio ato de jogar já é recompensador por si só, enquanto um loop mal concebido depende de estímulos externos — cutscenes, desbloqueios cosméticos, narrativa — para compensar a ausência de satisfação mecânica.

No caso de Void Descent, o game loop está estruturado em dois níveis complementares e interdependentes. O primeiro é o **loop micro**, que descreve a ação segundo a segundo dentro de cada sala e constitui o núcleo tátil da experiência — é o que o jogador *sente nas mãos*. O segundo é o **loop macro**, que descreve a progressão e a motivação do jogador ao longo de uma run completa e entre runs consecutivas — é o que o jogador *sente na cabeça*. A separação analítica entre esses dois níveis é fundamental porque cada um cumpre uma função distinta na experiência total: o micro precisa ser visceralmente satisfatório para manter o jogador engajado a cada instante, produzindo aquilo que a teoria de game design denomina *flow state* (estado de fluxo), enquanto o macro precisa fornecer propósito, direção e variabilidade suficientes para que esse engajamento não se esgote após as primeiras tentativas. Quando ambos os níveis funcionam em harmonia, o resultado é o fenômeno coloquialmente conhecido como “só mais uma run” — o jogador pretende parar, mas a combinação de satisfação imediata com curiosidade sobre o que a próxima dungeon vai gerar torna a interrupção psicologicamente difícil.

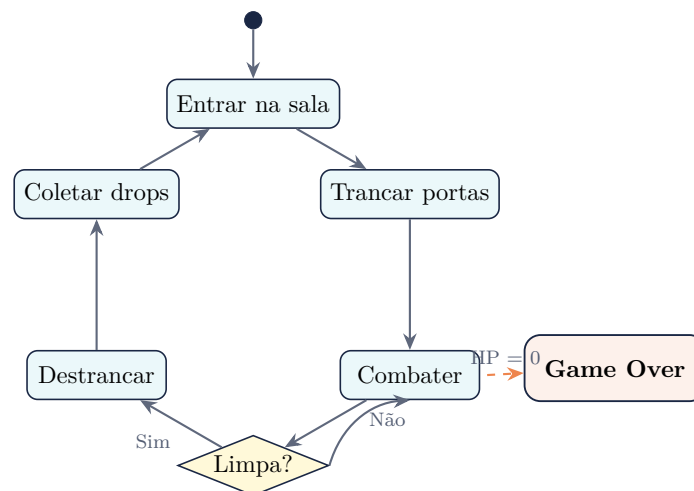
3.1. Loop Micro (segundo a segundo)

O loop micro é o ciclo de ação que o jogador repete a cada 5 a 10 segundos enquanto limpa uma sala. Ele define a sensação momento a momento do jogo — o chamado *game feel* — e, por essa razão, precisa ser intrinsecamente satisfatório para que toda a experiência se sustente. Se o ato básico de mover, mirar, atacar e desviar não for divertido por si só, nenhuma quantidade de conteúdo, progressão ou narrativa vai compensar essa deficiência. Em Void Descent, esse ciclo foi projetado segundo o princípio de *input-output imediato*: toda ação do jogador produz uma resposta visível e audível em menos de um frame, eliminando qualquer sensação de latência entre intenção e resultado. Essa responsividade extrema é o que distingue um combate que “parece bom” de um combate que o jogador apenas tolera.

O ciclo funciona da seguinte maneira:

1. **Entrada.** O jogador entra em uma sala. As portas trancam e os inimigos surgem em posições distribuídas pelo espaço.
2. **Processamento.** O jogador se movimenta pelo cenário (WASD), direciona a mira (mouse), realiza ataques (clique) e desvia de ataques inimigos (espaço para dash). Os inimigos reagem simultaneamente: alguns perseguem, outros atiram à distância, outros cercam. Cada tipo é previsível individualmente, mas o conjunto de vários tipos na mesma sala produz situações caóticas que exigem improvisação rápida.
3. **Feedback.** Cada ação do jogador e dos inimigos é acompanhada por respostas visuais e sonoras imediatas:
 - Quando um inimigo é atingido, ele exibe um flash branco, emite partículas neon e um som de impacto é reproduzido.
 - Quando o jogador é atingido, a tela inteira pisca brevemente em vermelho, um som grave ressoa e um coração desaparece da HUD.
 - Quando um inimigo é eliminado, uma explosão de partículas ocorre no local e um popup de score flutua na tela.
 - Quando todos os inimigos da sala são eliminados, as portas abrem com um som mecânico e há possibilidade de drop de item.

A combinação simultânea de mover, mirar, atacar e desviar faz com que o jogador nunca esteja parado; há tensão constante em cada segundo de gameplay, e essa tensão é o ingrediente essencial do *flow state*: o equilíbrio entre desafio e habilidade que mantém o jogador em um estado de concentração profunda e prazerosa. O dash com i-frames transfere integralmente a responsabilidade para o jogador: se ele morre, a culpa é sempre dele, nunca do jogo. Esse princípio, denominado *fair difficulty* na teoria de game design, é o que separa a dificuldade frustrante (onde o jogador sente que o jogo é injusto) da dificuldade motivadora (onde o jogador reconhece que poderia ter jogado melhor). Void Descent se posiciona deliberadamente no segundo caso: cada inimigo telegrafa seus ataques, cada projétil é visível e evitável, e o dash está sempre disponível desde que o cooldown tenha expirado. Não há mortes aleatórias nem situações impossíveis; há apenas execução insuficiente. Esse senso de agência absoluta, de que a sobrevivência depende exclusivamente da habilidade do jogador, é o que torna o loop micro intrinsecamente viciante e repetível: o jogador morre, mas sabe exatamente o que fez de errado, e esse conhecimento é o combustível que alimenta a próxima tentativa.

Figura 1 – Diagrama de atividades: Loop Micro (ciclo de combate por sala)

3.2. Loop Macro (minuto a minuto)

O loop macro responde a uma pergunta diferente e mais profunda: por que o jogador continua repetindo o loop micro? Ou seja, o que o motiva a entrar na próxima sala, descer ao próximo andar, e eventualmente começar uma nova run inteira após morrer e perder tudo? Em jogos lineares, a resposta costuma ser a narrativa — o jogador quer saber o que acontece a seguir. Em jogos com progressão permanente, a resposta costuma ser o acúmulo de poder — o jogador quer desbloquear a próxima habilidade ou item. Em Void Descent, nenhum desses mecanismos tradicionais existe: não há narrativa ramificada, não há upgrades permanentes entre runs, não há árvore de habilidades persistente. A motivação precisa emergir inteiramente da própria estrutura do loop, e isso ocorre através de três eixos que se retroalimentam.

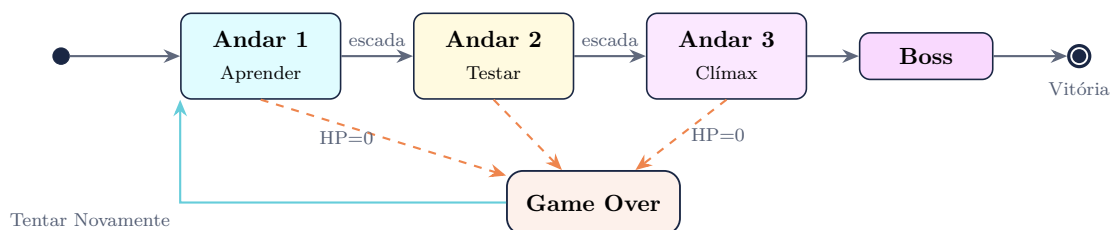
O primeiro eixo é o **ciclo de recompensa**. Ao limpar salas, o jogador acumula score e tem chance de receber drops na forma de armas ou consumíveis. Os baús garantem pelo menos um item por andar, funcionando como ponto de recompensa assegurada que reforça o comportamento de exploração e limpar salas por completo. Descer para um andar mais profundo introduz tipos novos de inimigos e, ao mesmo tempo, recompensas mais fortes, de modo que o risco e o retorno escalam juntos — um princípio clássico de game design conhecido como *risk-reward escalation*, que garante que a dificuldade crescente nunca pareça punitiva, pois é acompanhada de recompensas proporcionais. O score global alimenta a competição pessoal do jogador consigo mesmo (personal best), que é um motivador intrinsecamente poderoso mesmo na ausência de leaderboards online: o jogador não compete contra outros, mas contra a versão dele mesmo que jogou a run anterior. Cada nova pontuação recorde funciona como prova tangível de evolução, criando um ciclo positivo de auto-superação.

O segundo eixo é a **conversão de recursos**, que transforma cada item encon-

trado em uma decisão significativa. Armas encontradas no chão substituem a arma equipada, o que gera o dilema recorrente de “abro mão da minha arma rápida para pegar essa mais forte, porém mais lenta?”. Não existe resposta universalmente correta: a escolha ótima depende do andar em que o jogador está, dos inimigos que ele espera enfrentar, e de quanto HP lhe resta — uma arma ranged é mais segura quando o jogador está com pouca vida, mas uma melee com dano alto pode limpar salas mais rápido e reduzir o tempo de exposição. Como o slot de passivo também é único, o jogador enfrenta escolhas excludentes a cada item encontrado, e cada troca implica o abandono permanente do item anterior. Os consumíveis de cura devem ser guardados para momentos críticos, pois são raros, e usá-los cedo demais pode significar não tê-los disponíveis quando realmente necessário. E o HP é, em última instância, o recurso mais precioso e irrecuperável de todos: na ausência de regeneração passiva, cada ponto de vida perdido é um passo mais perto do permadeath, e cada poção de cura utilizada representa uma decisão com consequências que se propagam até o final da run. Esse emaranhado de trade-offs interconectados garante que o jogador esteja constantemente avaliando, comparando e decidindo, o que é o oposto de um gameplay mecânico e automático.

O terceiro eixo é a **progressão por run**, ilustrada no diagrama a seguir:

Figura 2 – Diagrama de atividades: Loop Macro (progressão da run)



O que mantém o jogador voltando, run após run, é a combinação específica de permadeath com geração procedural — dois elementos que, isoladamente, poderiam ser frustrantes, mas que juntos criam um ciclo de retenção excepcionalmente eficaz. Morrer dói, porque todo o progresso de itens acumulado é perdido definitivamente: a arma rara, o passivo que complementava o estilo de jogo, os corações preservados com tanto cuidado, tudo se vai. Porém, ao mesmo tempo, a dungeon nunca se repete: cada nova tentativa gera um layout diferente, com combinações distintas de inimigos, itens e salas, o que alimenta o pensamento de que “dessa vez vai ser diferente”. A imprevisibilidade procedural neutraliza a frustração do permadeath ao oferecer novidade constante como contrapartida da perda.

Esse ciclo é reforçado por um aspecto fundamental que constitui o verdadeiro motor do loop macro: embora o progresso de itens se perca entre runs, o **aprendizado do jogador não**. Os padrões de comportamento dos inimigos, os timings ideais para

usar o dash, as estratégias de gerenciamento de HP, a avaliação de quando trocar de arma e quando manter a atual, a leitura rápida de uma sala ao entrar nela — tudo isso persiste na mente do jogador entre runs como conhecimento tácito acumulado. É esse mecanismo que gera a sensação real de evolução pessoal, em oposição à simples evolução numérica de personagem que jogos tradicionais oferecem. Na run 1, o jogador morre no Andar 1 sem entender o que aconteceu; na run 5, já domina os Seekers e morre para os Shooters do Andar 2; na run 15, chega ao boss e descobre seus padrões; na run 30, completa o jogo inteiro e busca a pontuação perfeita. Essa curva de domínio, que emerge exclusivamente da repetição e do aprendizado ativo, é o que torna roguelikes como Void Descent inerentemente rejogáveis: o jogador não precisa de conteúdo novo para se sentir progredindo, porque o conteúdo que muda é ele próprio.

4. Narrativa e Ambientação

4.1. Sinopse

Sob a superfície do mundo digital existe o **Void**, um abismo procedural que se reconfigura a cada instante. Ninguém sabe quando ele surgiu nem o que o gerou, mas sua expansão corrompe tudo ao redor: dados se desfragmentam, estruturas colapsam, e entidades hostis emergem espontaneamente de padrões corrompidos. Para conter essa expansão, existem os **Divers**, operadores especializados que descem ao Void com uma única missão: alcançar o Núcleo no fundo do terceiro estrato e destruí-lo antes que o Void ultrapasse o ponto de contenção e comece a comprometer o mundo externo. O problema é que o Void nunca é o mesmo duas vezes; cada descida gera um labirinto completamente novo, com ameaças que evoluem e se reorganizam de formas imprevisíveis. A maioria dos Divers que entram não volta.

4.2. Personagens

O **protagonista** é o Diver, um operador representado visualmente por um sprite compacto com silhueta luminosa que emite luz ao redor. O Diver não fala e não possui história prévia; sua identidade é construída inteiramente pelas armas e passivos que o jogador coleta ao longo da descida. A cada nova run, o Diver entra no Void com equipamento básico e se transforma progressivamente conforme sobrevive e acumula novos itens. Essa abordagem de narrativa silenciosa transfere a construção de significado para o jogador, que projeta suas próprias motivações sobre o personagem que controla.

O **antagonista** é o Núcleo, a entidade central do Void responsável por gerar

e sustentar a corrupção. Ele se manifesta como uma entidade massiva e pulsante na arena final do Andar 3. O combate contra o Núcleo possui duas fases distintas: na primeira, ele dispara projéteis organizados em padrões rotativos que exigem timing preciso de dash; na segunda, ele abandona os projéteis, invoca Seekers como reforço para pressionar o jogador e avança diretamente em melee, testando tanto os reflexos quanto o gerenciamento de espaço.

4.3. Cenário

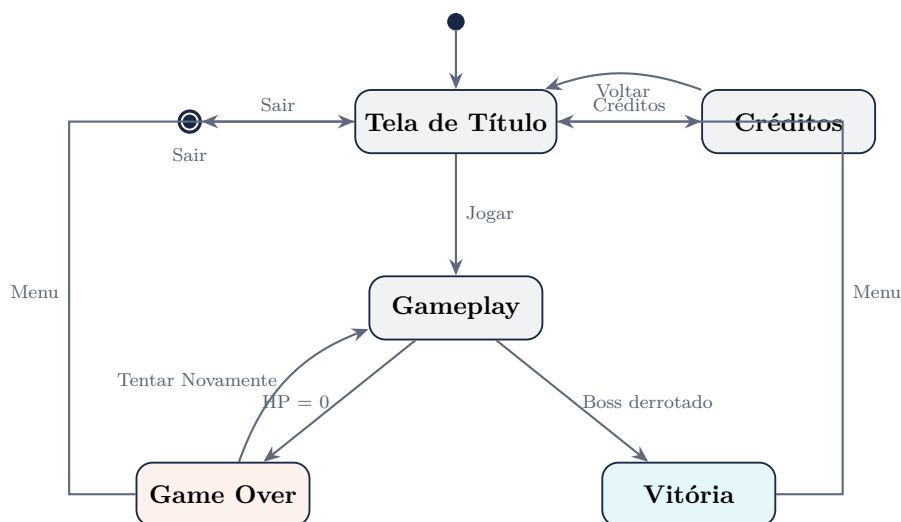
O Void é, visualmente, um espaço digital abstrato: fundo preto absoluto, com paredes e estruturas construídas como blocos geométricos coloridos em tons neon. Não há texturas realistas nem cenários orgânicos; tudo é construído com blocos e sprites sobre escuridão absoluta. Cada andar possui uma paleta de cor predominante que funciona como indicador da profundidade da corrupção. O Andar 1 utiliza tons de **ciano**, representando estruturas relativamente estáveis. O Andar 2 faz a transição para **amarelo e laranja**, sinalizando instabilidade geométrica crescente. O Andar 3 adota **magenta**, cor associada ao caos e à pulsação do Núcleo. Essa progressão cromática serve tanto para comunicar visualmente o nível de perigo ao jogador quanto para criar variação estética entre os andares sem necessidade de assets distintos.

5. Level Design (Esboço)

5.1. Fluxo de Telas

O fluxo entre telas segue o requisito “Menu ao Menu”, ou seja, o jogo sempre inicia na Tela de Título, transita para a gameplay quando o jogador escolhe “Jogar”, e após cada run oferece caminhos de volta ao menu principal tanto a partir da Tela de Vitória quanto da Tela de Game Over. Nesta última, o jogador também pode recomençar diretamente uma nova run sem passar pelo menu. Não existe nenhum estado do qual o jogador não consiga voltar ao menu. O diagrama a seguir ilustra todas as transições possíveis entre os estados do jogo.

Figura 3 – Diagrama de estados: fluxo de telas do jogo



5.2. Progressão de Dificuldade

A progressão de dificuldade foi dividida em três momentos distintos e crescentes, cada um correspondendo a um andar da dungeon. Essa estrutura garante que o jogo nunca jogue o jogador numa situação para a qual ele não foi preparado; cada andar introduz novos desafios de forma incremental.

1. Apresentação (Andar 1, paleta ciano). O primeiro andar é composto por 5 a 6 salas espaçosas, incluindo 1 baú. Apenas Seekers estão presentes, que são inimigos melee lentos cujo único comportamento é caminhar na direção do jogador em linha reta. Essa simplicidade é proposital e segue o princípio de design chamado *teaching through play*: em vez de exibir telas de tutorial com texto instrutivo, o jogo coloca o jogador diante de um inimigo cuja ameaça é óbvia e cuja solução é intuitiva — mover-se para longe e atacar. Ao sobreviver às primeiras salas, o jogador internaliza organicamente

as mecânicas fundamentais de movimentação e combate sem que o jogo jamais tenha pausado para explicitá-las. As salas espaçosas desse andar são intencionalmente amplas para proporcionar margem de erro generosa: o jogador ainda está aprendendo a navegar e precisa de espaço para errar sem ser punido de forma desproporcional. Os drops nesse andar são generosos, funcionando como recompensa imediata que reforça o comportamento de explorar e limpar salas, e que prepara psicologicamente o jogador para o Andar 2, onde a generosidade diminui de forma perceptível.

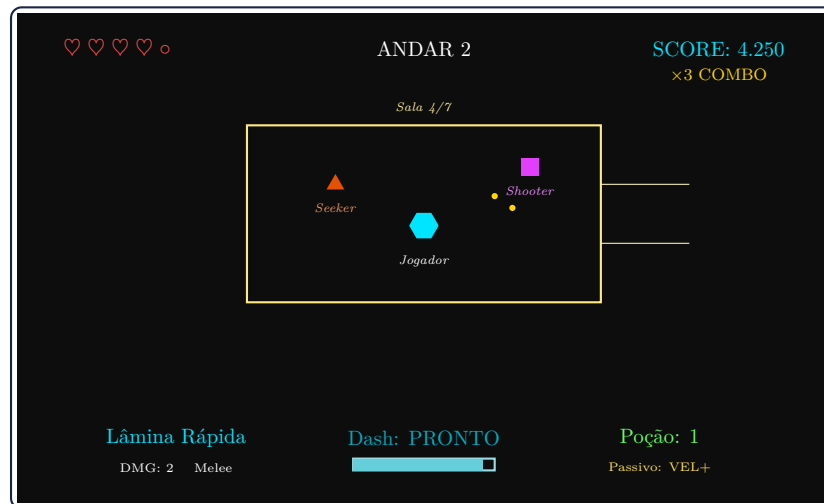
2. Teste (Andar 2, paleta amarela). O segundo andar contém 6 a 7 salas menores, incluindo 1 baú, e uma quantidade significativamente maior de inimigos por sala. Os Shooters aparecem pela primeira vez aqui, introduzindo projéteis inimigos e forçando o jogador a utilizar o dash de forma deliberada para desviar. A introdução dos Shooters é o primeiro ponto de inflexão da curva de dificuldade: até aqui, o jogador só precisava se afastar dos Seekers; agora, precisa lidar com ameaças que o alcançam a distância, o que exige uma mudança fundamental de postura — de simplesmente correr para longe, para ativamente esquivar usando os i-frames do dash. As salas menores criam pressão espacial adicional, pois há menos espaço para manobras evasivas, e a combinação de Seekers perseguindo em melee com Shooters disparando de longe força o jogador a priorizar alvos e tomar decisões de posicionamento sob pressão. É nesse andar que o jogador percebe que precisa de uma arma funcional e de um mínimo de estratégia para sobreviver. A maioria das runs tende a terminar no Andar 2, e essa taxa de mortalidade é intencional: ela marca a transição entre o tutorial implícito do Andar 1 e o desafio real que o jogo oferece, funcionando como o filtro que separa tentativas exploratórias de tentativas competitivas.

3. Clímax (Andar 3, paleta magenta). O terceiro e último andar possui apenas 3 salas regulares, incluindo uma sala de baú, seguidas pela arena final do boss. A brevidade desse andar é uma escolha de ritmo (*pacing*): após a extensão e a dificuldade crescente do Andar 2, o Andar 3 acelera deliberadamente em direção ao confronto final, criando um crescendo de tensão em vez de estendê-la indefinidamente. Os Bombers são introduzidos aqui como o último tipo de inimigo novo: eles pulsam visualmente como aviso e, após um delay telegrafado, explodem em área, causando dano significativo a tudo ao redor. Sua introdução tardia é proposital: o jogador já domina Seekers e Shooters e agora precisa integrar um terceiro tipo de ameaça ao seu modelo mental de combate, o que eleva a complexidade tática das salas finais. O Núcleo, boss final, é projetado para testar absolutamente tudo que o jogador aprendeu ao longo da run inteira. Na Fase 1, ele dispara projéteis em padrões rotativos que exigem timing preciso de dash, testando a capacidade de esquiva aprendida no Andar 2; na Fase 2, abandona os projéteis, invoca Seekers como reforço para pressionar o jogador e avança diretamente em melee, testando simultaneamente os reflexos, o gerenciamento

de espaço e a capacidade de priorizar alvos sob pressão máxima. A transição entre fases é abrupta e não anunciada, o que pune jogadores que se acomodaram a um único padrão e recompensa aqueles que mantiveram a adaptabilidade ao longo de toda a run.

5.3. Mockup da Tela

Figura 4 – Mockup da HUD durante gameplay



A HUD foi projetada segundo o princípio de *glanceability*: toda informação crítica deve ser compreensível em uma olhada de fração de segundo, sem exigir que o jogador desvie atenção do combate para interpretá-la. Ela exibe, de forma permanente e simultânea, o indicador de vida (corações, posicionados no canto superior esquerdo por serem a informação mais importante), o andar atual (centro superior, como referência de progressão), o score com multiplicador de combo (canto superior direito, atualizado em tempo real), a arma equipada com seus atributos (inferior esquerdo), o estado do dash com barra de cooldown visual (inferior central), o consumível disponível e o passivo ativo (inferior direito). A distribuição dos elementos obedece a uma hierarquia de urgência: informações de sobrevivência (HP, dash) estão nos cantos periféricos onde o olho as capta naturalmente, enquanto informações de performance (score, combo) estão em posições secundárias. Todos esses elementos permanecem na tela durante toda a fase, e cada mudança de estado é acompanhada por feedback visual imediato: quando o jogador perde vida, um coração desaparece da barra; quando coleta um item, o contador correspondente se atualiza instantaneamente; quando o combo aumenta, o multiplicador pulsa brevemente para chamar atenção. Essa transparência total de informação é uma decisão de design consciente e inegociável: em um jogo com permadeath, onde cada erro pode custar a run inteira, é essencial que o jogador tenha acesso constante e inequívoco ao seu estado atual para tomar decisões informadas a cada segundo.

6. Estética e Áudio (Direção de Arte)

6.1. Estilo Visual

O estilo visual de Void Descent pode ser descrito como **neon procedural**. Todo elemento visual que aparece na tela é gerado por código: a textura do jogo é um atlas gerado algoritmicamente durante a inicialização, onde cada tile (paredes, criaturas, itens) é desenhado por funções procedurais em vez de importado de um arquivo de imagem externo. O fundo é preto absoluto, e essa escolha não é apenas estética: o preto funciona como tela em branco que maximiza o contraste com os elementos neon que compõem o mundo, fazendo com que até os menores detalhes visuais sejam imediatamente perceptíveis.

As **paredes e estruturas** são renderizadas como blocos 3D com faces coloridas em tons neon sobre fundo preto, e a cor dessas superfícies varia de acordo com o andar (ciano no primeiro, amarelo no segundo, magenta no terceiro), criando identidade visual distinta para cada nível sem necessidade de tilesheets diferentes. O contraste entre as cores vibrantes e o preto do ambiente, combinado com a iluminação por vértice, produz naturalmente uma sensação de brilho neon mesmo sem um shader de glow dedicado. O **jogador** é representado por um sprite de operador com silhueta luminosa que funciona como fonte de luz pontual (iluminando os blocos ao redor), desenhado com proporções compactas e facilmente identificável mesmo em situações caóticas com muitas entidades na tela. Cada **tipo de inimigo** possui um sprite distinto cuja silhueta comunica seu comportamento antes mesmo do jogador interagir com ele: os Seekers são criaturas lentas com postura de avanço (sugerindo perseguição implacável), os Shooters são entidades mais robustas e estáticas que telegrafam seus disparos (sugerindo distância), e os Bombers são formas instáveis que pulsam e crescem antes de explodir (sugerindo expansão e área de efeito). O boss é uma entidade massiva com presença visual dominante, ocupando espaço significativo na arena.

Os **itens** aparecem como ícones brilhantes no chão, com a cor indicando o tipo: ciano para armas, verde para itens de cura, amarelo para passivos. Essa codificação cromática permite que o jogador identifique instantaneamente o que encontrou sem precisar parar para ler texto. O sistema de **partículas**, composto por pequenos sprites neon emitidos em rajada, acompanha cada impacto e cada morte de inimigo, reforçando o feedback visual e contribuindo para a sensação de “juice” que torna o combate satisfatório.

A **iluminação** é calculada por vértice no shader, com suporte a múltiplas luzes pontuais que atenuam com a distância. Cada fonte de luz (projéteis, explosões, o próprio

jogador) contribui para a iluminação local, enquanto as áreas distantes mergulham progressivamente em escuridão. Esse efeito cumpre dupla função: esteticamente, cria uma atmosfera tensa e claustrofóbica; mecanicamente, limita a visão do jogador e força uma postura mais cautelosa ao explorar salas. O pipeline visual inclui **fog** baseado em profundidade (que dissolve a geometria distante no preto, criando a sensação de abismo infinito) e **posterização** de cor no fragment shader (que quantiza os valores RGB em níveis discretos, produzindo uma estética retrô e digital coerente com a premissa do Void como espaço computacional corrompido).

6.1.1. Moodboard

O moodboard a seguir ilustra o território visual que Void Descent ocupa. As imagens foram selecionadas para demonstrar os princípios estéticos centrais do projeto: alto contraste neon-sobre-preto, paletas cromáticas por zona, iluminação dramática e feedback visual exagerado durante combate. Void Descent se diferencia por gerar todos esses elementos proceduralmente, sem nenhum asset pré-fabricado.

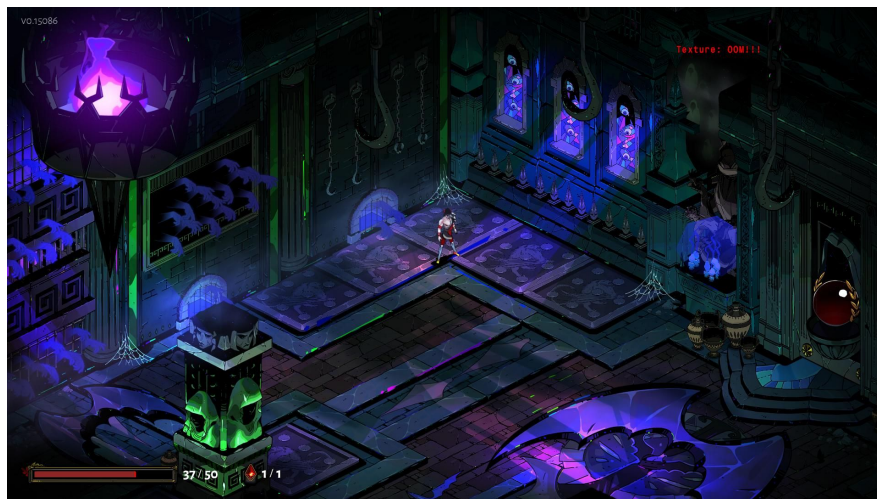


Figura 5 – Princípio de atmosfera: iluminação neon sobre fundo escuro, paleta roxa/ciano, HUD integrada ao cenário. Void Descent aplica esses princípios com blocos 3D e sprites gerados proceduralmente, sem assets externos.



Figura 6 – Princípio de combate: feedback visual intenso com partículas e claridade durante ação. Impactos geram contraste contrário ao fundo escuro, efeito que Void Descent reproduz com sprites procedurais e iluminação por vértice.



Figura 7 – Princípio de ambientação: paleta ciano/roxa com profundidade atmosférica. Void Descent utiliza transição cromática entre andares para comunicar progressão e perigo crescente.

6.2. Sonoplastia

Assim como a arte, o áudio de Void Descent é 100% procedural. Todos os sons do jogo são sintetizados em tempo real através da Web Audio API, utilizando uma biblioteca leve de síntese sonora que gera tanto efeitos quanto música a partir de descritores numéricos (frequências, envelopes, formas de onda), sem nenhum arquivo de som pré-gravado. Os buffers de áudio são gerados de forma assíncrona durante o carregamento do jogo e reproduzidos via `BufferSource` conectado ao `AudioContext`. Essa abordagem mantém consistência com a filosofia geral do projeto (zero assets externos) e também oferece vantagens práticas, como tamanho de build reduzido e possibilidade de variação paramétrica dos sons.

A **música** consiste em arpeggios synth ambient que mudam de tom conforme o andar em que o jogador se encontra. No Andar 1, os tons são agudos e limpos, transmitindo uma sensação de relativa segurança; conforme o jogador desce, a trilha se torna progressivamente mais grave, mais tensa e mais dissonante. A trilha do boss é especialmente distinta: possui BPM elevado e camadas adicionais de osciladores que se acumulam para criar uma sensação de urgência sonora.

Os **efeitos sonoros** são gerados com técnicas básicas de síntese de áudio. Ataques utilizam sweeps de frequência ascendentes, impactos utilizam noise bursts curtos, mortes de inimigos utilizam sweeps descendentes, o dash usa filtered noise com envelope rápido, e a coleta de itens é sinalizada por arpejos ascendentes de três notas. O resultado geral é um estilo sonoro que pode ser descrito como synthwave procedural, coerente com a estética visual neon do jogo inteiro.

7. Planejamento Técnico

7.1. Equipe

Void Descent é desenvolvido por uma equipe de 4 integrantes: Luis Gustavo Olímpio, Lauan Matheus, José Melquíades e João Leonardi. A divisão de trabalho contempla game design, programação, arte procedural e áudio. A proposta de zero assets facilita a coordenação da equipe, pois elimina a necessidade de pipelines separados para arte e som: todo o conteúdo visual e sonoro é implementado diretamente em código, permitindo que todos os integrantes contribuam no mesmo ambiente de desenvolvimento.

7.2. Engine e Tecnologia

Tabela 2 – Stack técnico do projeto

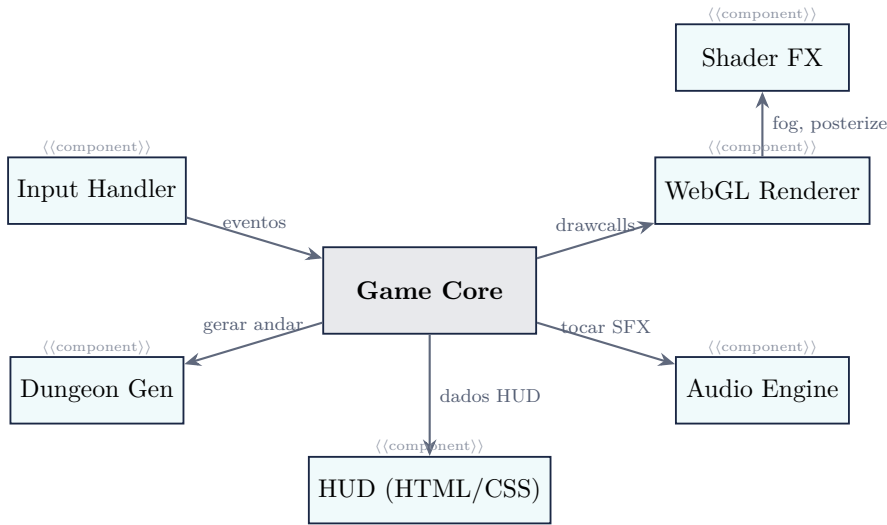
Componente	Tecnologia
Engine	Custom (sem engine comercial)
Linguagem	JavaScript vanilla
Renderização	WebGL com shaders GLSL para iluminação e efeitos visuais
Interface (HUD)	Elementos HTML/CSS sobrepostos ao canvas WebGL
Áudio	Web Audio API com biblioteca de síntese procedural
Build	Concatenação de módulos + minificação + Electron para .exe
Arquitetura	Escopo global, concatenação de arquivos, zero dependências

A decisão de construir uma engine custom, em vez de utilizar uma engine comercial como Unity ou Godot, foi tomada deliberadamente. Essa escolha demonstra domínio técnico completo sobre cada componente do pipeline: renderização, física, input e áudio. O código das mecânicas principais é inteiramente autoral, escrito do zero especificamente para este projeto. O WebGL foi escolhido como API gráfica porque fornece acesso direto a shaders GLSL, permitindo a implementação de iluminação por vértice com múltiplas luzes pontuais, fog por profundidade e posterização de cor, tudo com alta performance e controle total sobre o resultado visual. A arquitetura do código segue um modelo de escopo global com concatenação de arquivos: cada módulo (render,

entidades, áudio, geração de níveis) é escrito em um arquivo separado e concatenado num único bundle durante o build, que é então minificado para redução de tamanho. O Electron é utilizado exclusivamente na etapa final de empacotamento, para gerar o executável .exe exigido para entrega.

7.3. Arquitetura de Componentes

Figura 8 – Diagrama de componentes: arquitetura do sistema



7.4. Ferramentas de Apoio

Tabela 3 – Ferramentas utilizadas no desenvolvimento

Ferramenta	Uso
VSCode	Editor de código principal
Chrome DevTools	Debug, profiling de performance e shaders
Git / GitHub	Versionamento e backup do projeto
Electron Packager	Geração do executável .exe
Nenhum editor de arte	Todo o visual é gerado por código

7.5. Cronograma Macro

O desenvolvimento está planejado em 4 semanas, cada uma com entregas específicas e bem delimitadas. A divisão foi feita de modo que a primeira semana produza um protótipo jogável mínimo, e cada semana subsequente adicione camadas de complexidade até atingir o produto final:

Tabela 4 – Cronograma de desenvolvimento até a P2

Semana	Entregas
1	Core engine: renderer WebGL com shaders (iluminação, fog), movimentação do jogador, geração de dungeon (room placement), câmera top-down, colisão com paredes.
2	Gameplay: combate melee e ranged, 3 tipos de inimigo com AI, sistema de vida, dash com i-frames, itens (8 armas + consumíveis + passivos), pontuação.
3	Fluxo completo: Tela de Título, 3 andares com boss, Game Over, Vitória. HUD funcional. Áudio synth (SFX + música). Primeiro build Electron.
4	Polimento: partículas, fog, posterização, iluminação por vértice, feedback visual. Balanceamento de dificuldade. Testes de gameplay. Build final .exe.

7.6. Dificuldades Técnicas Previstas

1. **Geração de dungeon balanceada.** O algoritmo de room placement, por ser aleatório, pode eventualmente gerar layouts desconectados ou injustos. Para mitigar esse risco, o sistema aplica uma validação pós-geração via flood fill, que percorre o mapa e garante que todas as salas sejam alcançáveis a partir do ponto de spawn. Caso a validação falhe, o andar é regenerado.
2. **Performance em situações de pico.** Quando há muitos inimigos, projéteis e partículas ativos simultaneamente, o framerate pode cair. A mitigação planejada envolve o uso de object pooling para reaproveitar entidades já instanciadas, além de um limite máximo de partículas ativas na tela.
3. **Balanceamento de dificuldade.** Roguelikes são notoriamente difíceis de balancear: se o jogo é muito fácil, torna-se tedioso; se é muito difícil, torna-se frustrante. A estratégia de mitigação é centralizar todos os valores numéricos (dano, vida, cooldowns) em um arquivo de configuração separado, permitindo iterações rápidas de balanceamento através de playtesting.
4. **Empacotamento Electron.** Como esta é a primeira vez que a ferramenta será utilizada no projeto, existe risco de problemas de compatibilidade na geração do .exe. A mitigação é testar o processo de build já na Semana 3, de modo que haja tempo para resolver eventuais obstáculos antes da entrega final.

7.7. Posicionamento no Mercado

Do ponto de vista de mercado, Void Descent se insere no subgênero de roguelikes top-down em tempo real, um espaço que inclui títulos amplamente conhecidos como *The Binding of Isaac*, *Enter the Gungeon*, *Hades* e *Nuclear Throne*. Embora esses jogos compartilhem elementos fundamentais com Void Descent (combate em tempo real, geração procedural de níveis, permadeath), a proposta do projeto se diferencia de todos eles em aspectos técnicos e conceituais específicos. A tabela a seguir organiza essa comparação de forma visual.

Tabela 5 – Análise competitiva: Void Descent vs. mercado

Característica	Isaac	Gungeon	Hades	N. Throne	Void D.
100% procedural (sem assets)					•
Engine custom autoral					•
Visual neon procedural					•
Áudio sintetizado em tempo real					•
Runs curtas (<15 min)				•	•
Permadeath	•	•		•	•
Combate twin-stick		•		•	•

Como a tabela evidencia, o diferencial central de Void Descent é a abordagem **zero assets**. Toda a experiência, da arte aos mapas, dos inimigos ao som, é gerada exclusivamente por algoritmos, sem nenhum recurso externo. Enquanto os títulos estabelecidos listados na tabela dependem de equipes dedicadas de artistas, animadores e designers de som para produzir seus conteúdos visuais e sonoros, Void Descent demonstra que é possível produzir uma experiência coesa, coerente e visualmente impactante utilizando exclusivamente código. Além de funcionar como jogo, essa proposta também serve como prova de conceito técnica, exibindo domínio prático sobre WebGL, síntese de áudio procedural e técnicas de geração algorítmica aplicadas a múltiplos domínios simultaneamente.

A. Histórico de Revisões

Este apêndice registra as alterações feitas no GDD após a entrega original da P1, garantindo rastreabilidade entre versões do documento.

Versão	Data	Alterações
1.0	02/mar/2026	Versão inicial entregue na P1.
1.1	09/mai/2026	Revisão de coerência interna pré-entrega da P2: - Seção 2.3.4 (Geração): faixa de salas dos Andares 1 e 2 ajustada de “entre 5 e 8 salas cada” para “5 a 6 (Andar 1) e 6 a 7 (Andar 2)”, alinhando com os valores específicos da Seção 5.2. - Seção 2.3.4 (Geração): adicionada referência cruzada <code>\ref{sec:progressao}</code> para a seção de progressão detalhada. - Seção 2.3.2 (Vida): redacao sobre escassez de cura reescrita para clarear que a raridade é <i>progressiva por andar</i>, eliminando a tensão com a descrição de “drops generosos no Andar 1” presente na Seção 5.2. - Adição deste apêndice de histórico de revisões.

As alterações acima não mudam o escopo ou o conceito do jogo. Trata-se exclusivamente de *polimento textual* para eliminar pontos de fricção detectados em revisão sistemática. Toda divergência entre o que o GDD prescreve e o que o jogo entrega está documentada no Capítulo 8 da *Documentação Técnica* (`docs/main.pdf`), na seção “Mudanças de escopo em relação ao GDD”.